Project 2 – Acme Analytics Linux Platform

Day 7 · Phase 4 starts: where a slow application comes from, stopping a runaway process, pipes

Muhammad Bilal Javed · 24 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01, gw01 (powered on), svc01 (GLPI)
Status	Phase 3 ✅ (Day 6) · Phase 4 🟡 – Step 4.1 runaway process ✅ (concept, hands-on on app01, GLPI ticket #2) · Step 4.2 pipes ✅ (FIFO as reference)
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 6.

---

1. A question from industry – too many backups, and backup logs that grow too big

💡 **In plain words:** a friend who works as a Linux administrator asked: in a real company backups are taken all the time – what happens when the backups, and the backup **logs**, become huge? These are two different problems with two different fixes.

Problem	Industry fix	Already in this project
The backups grow	Retention (e.g. 7 daily, 4 weekly, 12 monthly) · incremental copies · deduplication (restic, borg) · move old copies to cheaper storage (AWS S3 → Glacier with a lifecycle rule)	`find … -mtime +7` keeps 7 days · `rsync --link-dest` hard links (Day 6)
The backup log grows	Log a summary, not every file (`rsync -v` writes one line per file) · rotate and compress logs with `logrotate` · journal size limit (`SystemMaxUse=`) · watch `/backups` and `/var/log` too	One `logger` line per run · logrotate comes in Phase 5

Interview line: *"I keep backups small with incremental copies, retention and cheaper storage for old copies, and I keep logs small by logging summaries and rotating them with logrotate."*

2. Real-world troubleshooting practice – how the incident drills fit in

💡 **In plain words:** to train like an on-call administrator, each phase will end with an **incident drill on my own Acme servers**: something realistic breaks, and I fix it like a real ticket – triage → hypotheses → commands → fix → verify → write-up in GLPI. The scenarios come from a hands-on troubleshooting lab (SadLab) and an L2/L3 incident workbook; only the cases that match this platform are used.

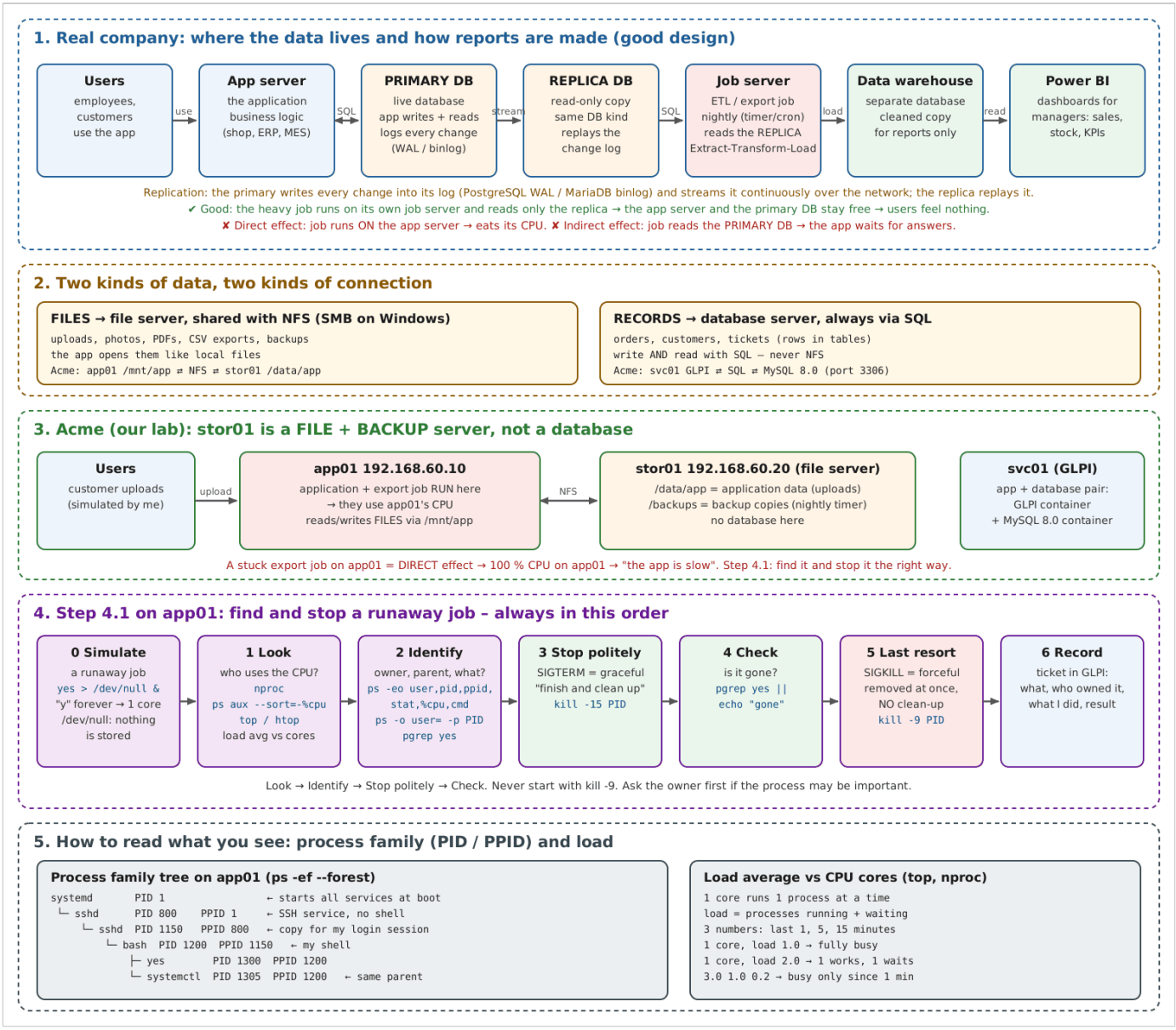
Phase	Drill
4 – Processes, signals	Runaway log writer · deleted file still filling the disk (`lsof +L1`) · "the server is slow"
5 – Logging	Backup log too big → logrotate · log rotation stalls a service
Backups	Backup "succeeds" but the restore fails → add a verify step to the nightly backup
6 – Security	SSH key login rejected · 403 Forbidden · firewall rule blocks a service
7 – Web service	Nginx won't start · works locally, not from outside
8 – Disaster drill	Read-only root after a power cut · wrong device mounted · inodes full

Several "advanced" workbook incidents were already solved for real in this project: a process hung on NFS after a storage failover (Day 5 stale mount and **D state**), an LVM snapshot during maintenance (Day 5), the wrong device mounted (why fstab uses **UUIDs**, Day 4), a backup that must be proven by a restore (Day 4 restore drill), and a cron job versus a timer (Day 6).

3. Phase 4 starts – Step 4.1: where a slow application comes from

💡 **The story:** app01 is Acme's application server. In real companies the most common daily problems are a **process that eats all the CPU**, a **job that hangs**, or a program that **won't stop properly**. Phase 4 teaches how to **find** the guilty process, **stop it the right way**, and finally **cap** what a service may use, so one bad program cannot freeze the whole server. Before touching commands, I worked out where such a process comes from in a real company and in Acme.

📸 Architecture – where the data lives, how reports are made (primary → replica → job server → data warehouse → Power BI), files vs records, Acme, the order of work on app01, the process family and the load average:



Step 4.1 architecture

Remember it in four lines:

```
Real company:  Users → App server ⇄ PRIMARY DB → (WAL/binlog stream) → REPLICA → Job server (ETL) → Data warehouse → Power BI
Replication:   the app writes SQL only to the primary; the primary logs every change and streams the log to the replica, which replays it
Acme:          Users → app01 (runs the app) ⇄ NFS (files) ⇄ stor01 (stores files + backups)  ·  svc01: GLPI ⇄ SQL ⇄ MySQL 8.0
Step 4.1:      Simulate → Look → Identify → Stop politely → Check → (kill -9 last) → Record
```

All on **app01**:

```
sudo apt install -y htop stress-ng
nproc                                                    # how many CPU cores
yes > /dev/null &                                        # simulate a runaway job
ps aux --sort=-%cpu | head -5                            # who uses most CPU
top                                                      # load average, %CPU, STAT – q to quit
htop                                                     # same, clearer – q to quit
ps -eo user,pid,ppid,stat,%cpu,cmd --sort=-%cpu | head -5  # owner, parent, state, command
ps -o user= -p $(pgrep yes)                              # who owns it
ps -ef --forest | less                                   # the process family tree
kill -15 $(pgrep yes)                                    # SIGTERM – graceful stop
pgrep yes || echo "gone"                                 # check
```

Term	Meaning
Runaway job	Busy and out of control – 100 % CPU, never finishes
Hung job	Stuck waiting – almost 0 % CPU, never finishes
Load average	Processes running + waiting for the CPU; 1-, 5- and 15-minute averages, compared with `nproc`
PID / PPID	The process's own number / the number of the process that started it
SIGTERM (`kill -15`) / SIGKILL (`kill -9`)	Graceful stop with clean-up / forceful stop, no clean-up – last resort
Primary / replica	Live database / read-only copy of the same kind, kept up to date by streaming the change log
WAL / binlog	The change log of PostgreSQL / MariaDB – the source of replication

3.1 The hands-on run on app01

Issue – `apt install` hangs at `0% [Working]`

Seen	On app01, `sudo apt install -y htop stress-ng` printed `Ign:` lines again and again and stayed at `0% [Working]`.
Cause	**gw01 was powered off.** gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT) – a **single point of failure**.
Fix	Powered on gw01; the install worked at once.
Lesson	When a server "cannot reach the internet", check the path hop by hop: `ip route` → ping the gateway → ping 8.8.8.8 → ping a name (DNS).

📸 **Evidence:**

```
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
0% [Working]_
```

apt hangs

📸 **Evidence – 2 CPU cores; the runaway job started as job [1], PID 1918:**

```
bilal_002@app01:~$ sudo apt install -y htop stress-ng
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
htop is already the newest version (3.0.5-7build2).
stress-ng is already the newest version (0.13.12-2ubuntu1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0
  libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager tcl tcl8.6 usb-modeswitch
  usb-modeswitch-data
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
bilal_002@app01:~$ nproc
2
bilal_002@app01:~$ yes > /dev/null &
[1] 1918
bilal_002@app01:~$ jobs
[1]+  Running                 yes > /dev/null &
```

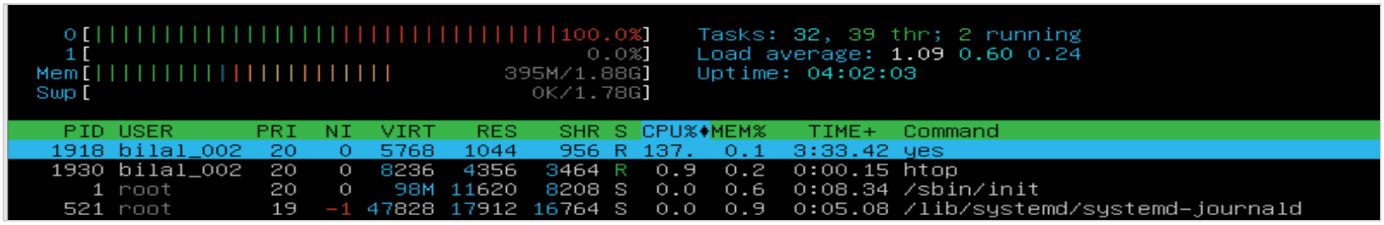
nproc yes

📸 **Evidence – Look: `yes` at the top with 99.9 % CPU, state R, owner `bilal_002` (shown cut as `bilal_0+`); without the minus in `--sort=-%cpu`, ps sorts lowest first:**

```
bilal_002@app01:~$ ps aux --sort=-%cpu | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
bilal_0+    1918 99.9  0.0   5768  1044 tty1     R    18:57   2:40 yes
root         711  0.5  0.4 314816  9312 ?        Ssl  14:59   1:15 /usr/bin/vmtoolsd
root        1557  0.3  0.0      0     0 ?        I    16:27   0:32 [kworker/0:0-events]
root        1550  0.2  0.0      0     0 ?        I    16:27   0:25 [kworker/1:1-events]
bilal_002@app01:~$
```

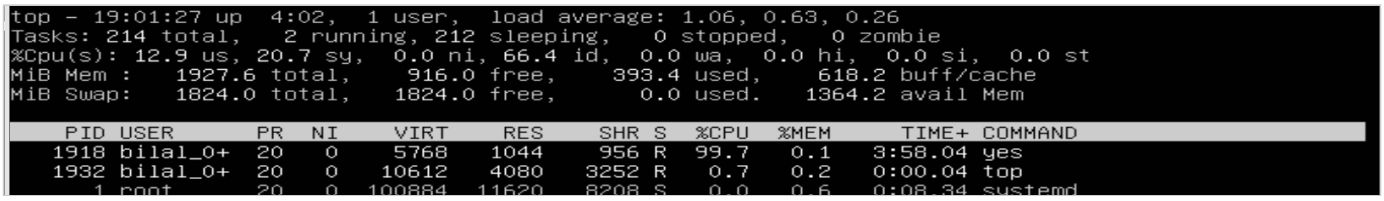
ps aux

📸 **Evidence – `htop`: core 0 at 100 %, core 1 at 0 % – `yes` fills exactly one of the two cores; load average 1.09:**



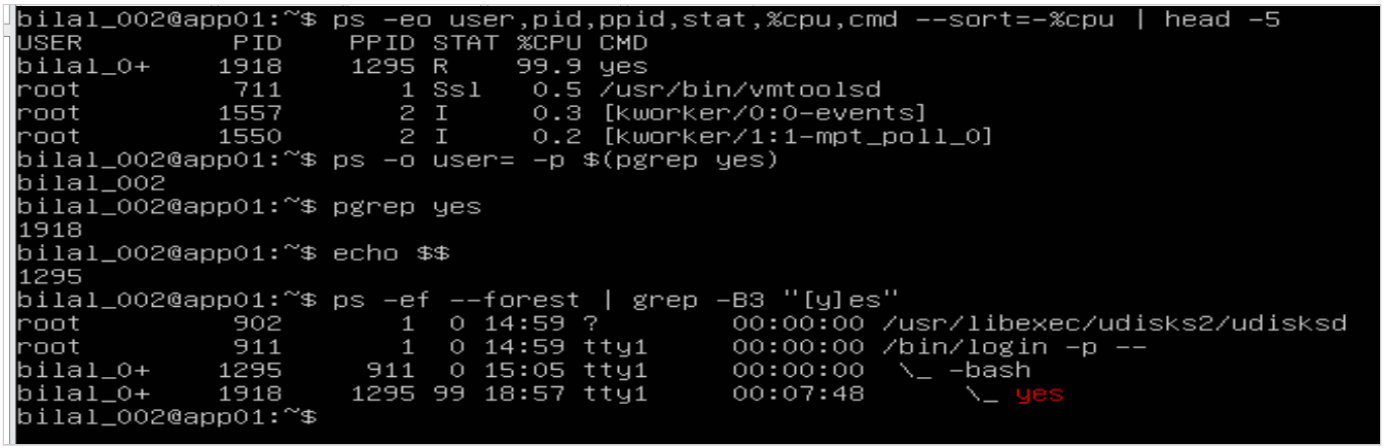
htop

📸 **Evidence – `top`: load average `1.06, 0.63, 0.26` – about 1.0 over the last minute and still rising over 5 and 15 minutes; `2 running, 0 zombie`:**



top

📸 **Evidence – Identify: PID 1918, PPID 1295; `echo $$` = 1295, so the parent is my bash; the family tree: `/bin/login` (console tty1) → `-bash` → `yes`. Over SSH, the parent of bash would be `sshd`:**



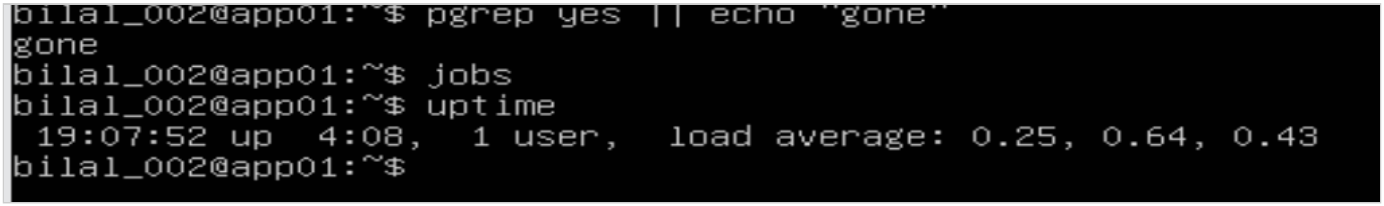
identify

📸 **Evidence – Stop politely with SIGTERM:**



kill 15

📸 **Evidence – Check: "gone", no jobs left, the load average falling (`0.25`, 0.64, 0.43 – the 1-minute value drops first):**

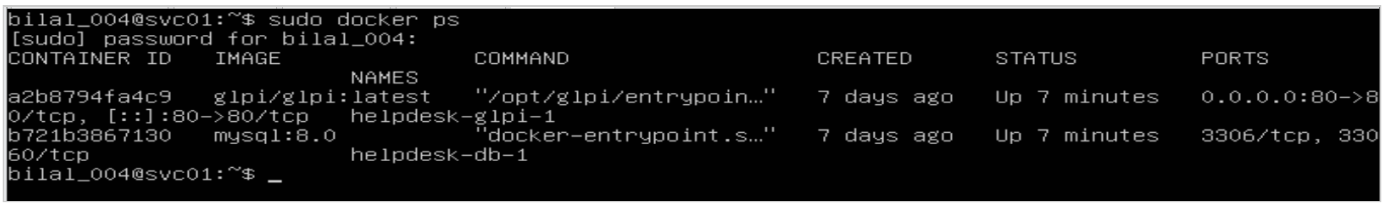


gone

Lesson: read the output **above** a success message. When `pgrep` was typed without a name it failed with an error – and `||` printed "gone" anyway. Only `pgrep yes || echo "gone"` is real proof.

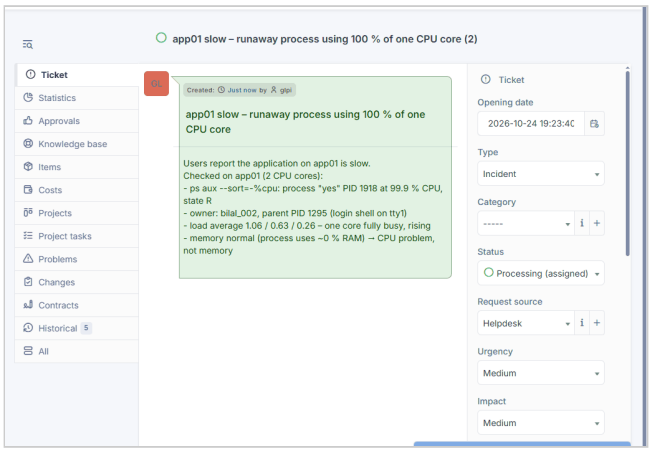
3.2 Record – GLPI ticket #2

📸 **Evidence – GLPI runs as an app + database pair on svc01: `glpi/glpi` publishes port 80 to the network; the database `mysql:8.0` shows only `3306/tcp` – not published, reachable only by GLPI inside Docker:**



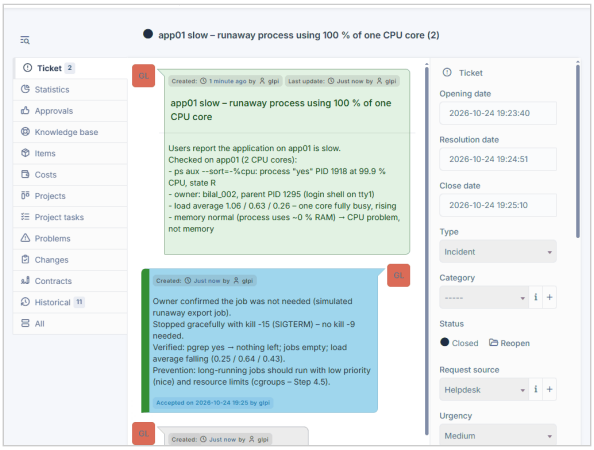
docker ps

📸 **Evidence – Incident ticket #2 created (Type Incident, Urgency/Impact Medium):**



ticket created

📸 Evidence – solution added and approved, ticket closed (opened 19:23:40, solved 19:24:51, closed 19:25:10 UTC):



ticket closed

Lesson: in a ticket, **Requester = who** reported it (a person), **Items = what** is broken (a computer from the inventory). In a company everyone works under a personal named account, not the built-in `glpi` super-admin.

Key understanding – the lab result (in my own words)

On app01 I first installed `htop` and `stress-ng`. The first attempt hung at `0% [Working]` with repeated `Ign:` lines because **gw01 was switched off**: gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT), so it is a single point of failure, and after powering it on the install worked. `nproc` showed **2 cores**. I created a runaway job with `yes > /dev/null &` (job [1], PID 1918). `ps aux --sort=-%cpu | head -5` put `yes` at the top with 99.9 % CPU, owner `bilal_002` (shown as `bilal_0+` because the name was cut off), state `R` (running), almost no memory (a CPU problem, not a memory problem) and a growing CPU TIME; without the minus in `--sort=-%cpu`, ps sorts lowest first and shows only idle processes. `htop` showed one core at 100 % and the other at 0 %, so `yes` fills exactly one of the two cores. `top` showed a load average of `1.06, 0.63, 0.26`: about 1.0 over the last minute (one core fully busy) and still rising over 5 and 15 minutes. `ps -eo user,pid,ppid,stat,%cpu,cmd` showed PID 1918 with PPID 1295, and `echo $$` printed 1295, so the parent of `yes` was my bash. `ps -ef --forest` showed the whole family: systemd (PID 1) → `/bin/login` (911, because I worked on the VMware console tty1; over SSH it would be `sshd`) → `-bash` (1295) → `yes` (1918). I stopped it gracefully with `kill -15` (SIGTERM): bash reported `[1]+ Terminated`, `pgrep yes || echo "gone"` printed "gone", `jobs` was empty, and `uptime` showed the load falling to `0.25, 0.64, 0.43`, the 1-minute value dropping first. I also learned to read the output above a success message: when I mistyped `pgrep` without a name, it failed with an error, and `||` printed "gone" anyway. Finally I recorded the incident in GLPI as ticket #2 – Type Incident, Requester = the person who reported it, Items = the computer that is broken – with the findings, the solution and a prevention note (low priority with `nice`, limits with cgroups), and closed it. `docker ps` on svc01 showed that GLPI is a real app + database pair: the `glpi/glpi` container publishes port 80, while its **MySQL 8.0** database shows only `3306/tcp`, not published, so users reach the app and only the app reaches the database.

Key understanding – the concept (in my own words)

1. The problem. The most common complaint about an application server is *"the application is slow"*. Very often the cause is one program using all of the server's CPU. To fix it, I must understand where the data lives, which job can cause the problem, and how to find and stop that job safely.

2. Where the data lives: files and records. Applications use two kinds of data. **Files** (uploads, photos, PDFs, CSV exports, backups) are kept on a **file server** and shared over the network with **NFS** (SMB on Windows); the application opens them as if they were on its own disk. **Records** (orders, customers, tickets)

are rows in tables inside a **database**, such as MariaDB or PostgreSQL. The application always talks to a database with **SQL**, both to write ("save this order") and to read ("show me today's orders"), and never with NFS. A server whose main job is running database software is called a **database server**. One database program can hold many separate databases (for example `shop`, `hr` and `glpi`), and two different database programs can even run on the same server, because each uses its own port (MariaDB 3306, PostgreSQL 5432). Most applications come as an **app + database pair**: the app contains the pages, buttons and **business logic** (the rules of what happens, for example "when an order is placed, reduce the stock"), and the database stores the data.

3. How a real company makes reports, and how it goes wrong. Users work in the application (a web shop, ERP or MES) on the **application server**. It writes and reads the live data in the **primary database**, always with SQL and only on the primary. The primary first writes every change into its **change log**, a numbered diary on disk (PostgreSQL: the **WAL**, write-ahead log; MariaDB: the **binlog**), and then changes the table. It streams that log **continuously over the network** to a **replica**, a read-only copy of the **same kind** of database (PostgreSQL → PostgreSQL, MariaDB → MariaDB), which **replays** the same changes on its copy, normally less than a second behind. This is **replication**. Every night an automatic **ETL job** (Extract, Transform, Load), started by a timer or cron, runs on a separate **job server**. It **extracts** millions of rows from the **replica**, **transforms** them (cleans and combines them) and **loads** them into a **data warehouse**, a separate database used only for reports. Moving data between different kinds of database like this is ETL, not replication. **Power BI** reads the data warehouse and shows dashboards to managers, such as sales, stock and KPIs. Typical jobs: *"export today's orders to a CSV file for accounting"*, *"create the monthly sales report"*, *"send new customers to another system every hour"*. This design keeps the heavy job away from the live system, so users feel nothing. If a company skips it, the heavy job slows the application in one of two ways. With a **direct effect**, the job runs **on the application server itself** and uses its CPU. With an **indirect effect**, the job reads the **primary database**, which becomes so busy that the application waits for its answers. A replica protects performance but is **not a backup**: if someone deletes data by mistake, the deletion is copied to the replica within seconds as well.

4. The Acme platform. Acme is smaller. **stor01 is a file and backup server, not a database server**: it holds `/data/app` (the customer uploads, on the LVM volume `lvdata`) and `/backups` (copies made by the nightly backup timer). app01 reads and writes these files through the NFS mount `/mnt/app`. The application and any export job **run on app01**, so a stuck job there is a **direct effect**: it uses app01's CPU, and the application becomes slow. Acme's real app + database pair is GLPI on **svc01**: every ticket, user and inventoried computer is a row in its **MySQL 8.0** database (a close relative of MariaDB), and GLPI reads and writes them with SQL. Only GLPI's port 80 is published to the network; the database port 3306 is reachable only by the GLPI container inside Docker, so users reach the app and only the app reaches the database.

5. Runaway job, hung job and the load average. A job causes trouble when it is **stuck in a loop**. A loop is a part of a program that repeats the same steps, such as "read the next row, write it, repeat", until it reaches its stop point, for example the last row. Because of a bug, the stop point never comes, so the job repeats forever at 100 % CPU, like a nightly export still running at 10 a.m. That is a **runaway job**: busy and out of control. A **hung job** is different: it uses almost no CPU because it is stuck **waiting**, like my Ctrl+Z waiting on NFS in Phase 3. A normal scheduled job, like my nightly backup, starts, finishes and stops. One **CPU core** runs one process at a time, and `nproc` shows how many cores a server has. The **load average** is the number of processes that want the CPU at the same moment, both running and waiting. `top` shows it as three numbers, the averages over the last **1, 5 and 15 minutes**. With 1 core, a load of 1.0 means fully busy, and 2.0 means one process is working while another waits. `3.0, 1.0, 0.2` means "busy only since a minute ago"; `3.0, 3.0, 3.0` means "busy for at least 15 minutes".

6. Processes, PID and PPID. Every running program is a **process** with its own number, the **PID**. The **PPID** is the PID of the process that **started** it, its parent. Processes form a family tree, shown by `ps -ef --forest`. **systemd** (PID 1) starts all services at boot, so services such as `sshd` have PPID 1 and no shell. When I log in over SSH, sshd starts a copy of itself for my session, and that copy starts my **shell**, bash, the program that reads the commands I type and runs them. Every command I start, for example `yes` or `systemctl restart sshd`, gets its **own PID**, and its **PPID is my bash**. Processes started by the same bash therefore share the same PPID.

7. Finding and stopping the job on app01, always in this order. Simulate → Look → Identify → Stop politely → Check → (kill -9 only as a last resort) → Record. Simulate: I first installed `htop` (a clearer live process viewer) and `stress-ng` (a test tool that creates CPU, memory or disk load on purpose, used later for limits). `yes > /dev/null &` creates a runaway job: `yes` prints the letter "y" without end, so it alone uses one full core; `>` sends this output to `/dev/null`, a special file where everything disappears and nothing is stored (it is not the `/bin` folder, which holds programs); and `&` runs it in the background. **Look:** `nproc` shows the number of cores, and `ps aux --sort=-%cpu | head -5`, `top` and `htop` list the processes using the most CPU. `yes` appears at the top with about 100 %, and the load average rises. **Identify:** `ps -eo user,pid,ppid,stat,%cpu,cmd` shows the **owner** (the user who started it), the PID, the PPID, the state (`R` = running), the CPU use and the **command** (`cmd`, the command line that started it, which shows what the program is). `ps -o user= -p $(pgrep yes)` shows only the owner, and `pgrep yes` shows only the PID. **Stop politely:** `kill -15` sends **SIGTERM**, a **graceful** stop: the program is asked to finish and clean up. **Check:** `pgrep yes || echo "gone"` prints "gone" when nothing is left (`||` means "if the first command finds nothing, run the second"). **Last resort:** `kill -9` sends **SIGKILL**, a **forceful** stop. The kernel removes the process at once and its clean-up never runs, so leftovers such as a lock file or a half-written file can remain; I use it only if a process ignores SIGTERM. **Record:** in a company, before stopping anything, I check who owns the

process and whether it is important, asking the owner if needed. Afterwards I write a GLPI ticket: what happened, who owned the process, what I did and the result.

4. Step 4.2 – pipes: output → input → output

💡 **The story:** a pipe (`|`) passes the output of one program straight into the next as its input – no file in between. Admins use it every day, and it saves real disk space when moving big data: compress and send in one go, instead of first writing a temporary archive.

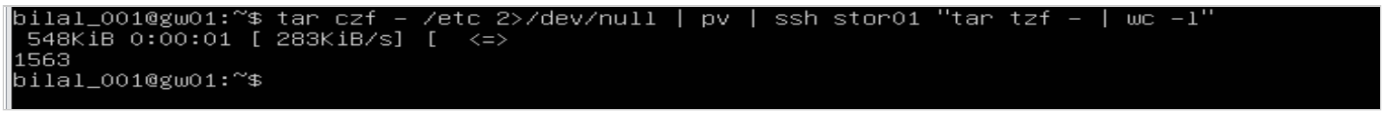
```
command 1 ──output──► | ──input──► command 2 ──output──► | ──input──► command 3 ──output──► screen
```

4.1 Hands-on – stream a compressed copy to another server (gw01 → stor01)

```
# gw01
sudo apt install -y pv
tar czf - /etc 2>/dev/null | pv | ssh stor01 "tar tzf - | wc -l"
```

Piece	Meaning
`tar czf - /etc`	**c**reate an archive, g**z**ip-compress it, **f**ile `-` = the pipe instead of a file
`2>/dev/null`	Hide the "permission denied" messages for root-only files
`pv`	**Pipe viewer** – shows the amount, speed (and % / ETA when the size is known)
`ssh stor01 "…"`	Carries the stream over the network and runs the command on stor01
`tar tzf - | wc -l`	On stor01: lis**t** what arrives and count the lines (`tar xzf - -C /data` would unpack = a real copy)

📸 **Evidence – 548 KiB streamed in 1 s; stor01 counted 1,563 files; nothing was stored on either server. `pv` shows `<=>` instead of % because a stream's size is not known in advance:**



tar pv ssh

Everyday pipe chains (`a | b | c`):

```
ps aux | sort -k3 -nr | head -3          # top 3 CPU users
grep bash /etc/passwd | wc -l            # how many users have a bash shell
pv shop.sql.gz | gunzip | mysql shop     # restore a dump with progress, no unpacked temp file
```

`|` (one bar) = pipe. `||` (two bars) = "or": run the second command only if the first fails – a different thing.

4.2 Reference only – named pipe (FIFO)

A FIFO is a **pipe with a name** – a tube between two programs **on the same server**. It **stores nothing** (size 0, type `p` in `ls -l`), data is gone once read (first in, first out), and the writer **waits** until a reader is there. It is needed only when two programs **cannot** be joined with `|` in one command line – a tool that accepts only a file name, or two programs started separately. Rare in daily admin work (examples: the Nagios/Icinga command file, some database export tools), so this part was kept as a reference and not run.

```
mkfifo /tmp/jobs                                                  # create the named pipe
ls -l /tmp/jobs                                                   # prw-r--r-- … 0 … /tmp/jobs
while read job < /tmp/jobs; do echo "processing: $job"; done      # terminal 1: the reader (worker)
echo "build-report" > /tmp/jobs                                   # terminal 2: the writer
rm /tmp/jobs                                                      # remove it

# the same transfer as above, as two separately started parts
mkfifo /tmp/stream
tar czf /tmp/stream /etc &                                        # writes "a file" — really the pipe
pv /tmp/stream | ssh stor01 "tar tzf - | wc -l"                   # reads it
rm /tmp/stream
```

Key understanding (in my own words)

A **pipe** (`|`) makes the output of one command the input of the next – **output → input → output** – with no file in between, like an assembly line, and only the last output reaches the screen. In `ps aux | sort -k3 -nr | head -3`, `ps` lists all processes, `sort` orders them by CPU and `head` keeps the top three. In real work pipes save time and disk space: on gw01, `tar czf - /etc | pv | ssh stor01 "tar tzf - | wc -l"` packed and compressed `/etc` into the pipe (`tar` packs many files into one stream: **c** create, **x** extract, **t** list, **z** gzip, **f -** = the pipe instead of a file), `pv` (**pipe viewer**) showed how much data passed and how fast (548 KiB in 1 second; a moving `<=>` instead of a percentage, because a stream's size is not known in advance), `ssh` carried the stream to stor01, and there `tar tzf -` listed what arrived and `wc -l` counted 1,563 files – nothing was stored on either server. With `tar xzf - -C /data` instead, stor01 would have

unpacked it, which is a real copy to a new server without any temporary archive; the pipe saves the temporary copy, but the target still needs space for what is finally stored there. `pv` is also used to show progress and time left for long jobs like restoring a database dump (`pv shop.sql.gz | gunzip | mysql shop`). One bar `|` is a pipe; two bars `||` mean "or" – run the second command only if the first fails. A **named pipe (FIFO)** is a pipe with a name in the filesystem: a tube between two programs on the same server that stores nothing (size 0, type `p` in `ls -l`), hands data over in order and only once, and makes the writer wait until a reader is there. It is needed only when two programs cannot be joined with `|` in one command line – for example a tool that accepts only a file name, or two programs started separately. It is rare in daily admin work, so I kept it as a reference: when I see a `p` at the start of an `ls -l` line, I know it is a pipe, not a normal file.

5. Next steps

1. VMware snapshot `40-automation` if not done yet.
2. Steps 4.3–4.5: signals and `trap` (why `kill -9` leaves a stale lock file), zombie processes, cgroups (`MemoryMax`, `CPUQuota`) – then the first incident drill.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation's *Connect to Server*.